bs_freshness_queue 060 — guard-freshness drain order, most-reopened-first (T)

`registry.json` carries 5 toy guards, `fingerprint.txt` carries the current artifact fingerprint (`fp-2026-09-29`) the freshness-queue is computed against.

guard_id	re-opens_count	fingerprint matches cur-rent?	last_ver-dict_at
GUARD-HIGH	9	no (`fp-OLD-1`) — STALE	2026-01-01
GUARD-MID	4	no (`fp-OLD-2`) — STALE	2026-06-01
GUARD-TIE	4	no (`fp-OLD-3`) — STALE	2026-08-01
GUARD-NEV-ER	2	no (never executed) — STALE	(absent)
GUARD-FRESH	99	**yes** — FRESH	2026-09-29

Per constitution SS11.4.226 ("... the never-executed/stale/over-budget set feeds a STANDING risk-ordered re-run queue (most-reopened-first SS11.4.189, then stalest-first)") the drain order the real `backstop.sh freshness-queue` subcommand MUST emit is:

1. `GUARD-HIGH` (reopens=9 — highest among the stale set)
2. `GUARD-MID` (reopens=4, `last_verdict_at=2026-06-01` — older/staler than GUARD-TIE)
3. `GUARD-TIE` (reopens=4, `last_verdict_at=2026-08-01` — the tie-break)

4. `GUARD-NEVER` (reopens=2 — lowest reopens_count wins last, DESPITE being maximally stale/never-executed: reopens_count is the PRIMARY sort key, staleness is only a tie-break on equal reopens_count)

`GUARD-FRESH` is **excluded from the queue entirely**, despite having the single highest `reopens_count` (99) of all 5 guards — because its stored `last_verdict_fingerprint` matches the current fingerprint (it already has a verdict on the current artifact and needs no re-run). This is the fixture's negative-control half: a naive implementation that ranks by `reopens_count` alone with no freshness gate would wrongly place `GUARD-FRESH` at rank 1.

`expected` : first line `QUEUE`, then the 4 stale guard_ids in required drain order, then `FRESH_EXCLUDED=GUARD-FRESH`, then the `FRESH_COUNT=1` / `STALE_COUNT=4` summary lines.

This fixture is proven non-vacuous by `guard_freshness_ref.py` (this project's own from-scratch reference implementation) in the RED test's Section C, BEFORE any claim is made about the absent real tool (`gates/backstop.sh freshness-queue`) in Section D. Section C also proves TWO discriminating control needles this data set is specifically built to exercise: (1) sorting by staleness alone (ignoring reopens_count) would produce a DIFFERENT order (`GUARD-NEVER` first, since it is maximally stale) — proving reopens_count genuinely dominates the primary sort; (2) sorting by reopens_count alone (ignoring the freshness gate) would wrongly INCLUDE `GUARD-FRESH` at rank 1 — proving the freshness gate is a real, necessary, separate mechanism from the ranking.